Curs 6

Adresarea datelor

Dupa cum a fost explicat in Capitolul 4, operanzii utilizati de diversele unitati functionale ale caii de date pot proveni din interiorul procesorului cat si din exteriorul acestuia. Notiunea de operanzi se refera atat la datele de intrare procesate cu o instructiune, cat si datele de iesire ce rezulta in urma executarii unei instructiuni. **Prin adresare se inteleg mijloacele prin care sunt specificate locatiile operanzilor.**

Exista mai multe tipuri de adresare (denumite si *moduri de adresare*); de exemplu, operanzii pot fi obtinuti direct dintr-un registru sau de la o adresa de memorie interna sau externa. In acest capitol vom trece in revista modurile de adresare disponibile la procesoarele DSP, punand accentul pe modurile cele mai folosite in aplicatii.

Multe procesoare poseda doar un subset al modurilor de adresare prezentate in continuare, cu diverse nivele de flexibilitate si performanta. In plus, pot exista restrictii la folosirea unui anumit mod de adresare. In multe cazuri, un anumit mod poate fi utilizat doar cu un subset al setului de instructiuni implementat de procesor. In cazuri extreme, un mod de adresare specific poate fi disponibil doar pentru o instructiune. Unele familii de procesoare de semnal folosesc o *arhitectura load-store*. Aceasta inseamna ca procesoarele folosesc instructiuni speciale pentru transferul datelor cu memoria. Restul instructiunilor pot acesa date doar din registri.

Modul de adresare a datelor este indicat, fie explicit in codul instructiunii fie este implicit. Este des intalnit cazul in care o instructiune combina doua sau mai multe moduri de adresare, folosind moduri diferite pentru operanzi diferiti.

Majoritatea procesoarelor DSP poseda una sau mai multe unitati de generare a adresei (AGU – Address Generating Units) care sunt dedicate calcului adreselor. Producatorii folosesc diverse nume pentru acestea. De exemplu, Analog Devices foloseste denumirea de AGU (Address Generating Unit) pentru date, iar AT&T denumeste acest modul unitate de control aritmetica. Un AGU poate efectua una sau mai multe operatii complexe de calcul al adreselor pe ciclu de tact fara a utiliza calea de date principala a procesorului. In acest fel calculul adreselor poate avea loc in paralel cu operatiile aritmetice efectuate asupra datelor, marind performanta procesoarelor. Diferentele intre unitatiile de generare a adreselor se manifesta prin modurile de adresare oferite si prin capabilitatiile si flexibilitatea fiecarui mod de adresare. In continuare vor fi discutate in detaliu modurile de adresare disponibile la procesoarele digitale de semnal.

6.1 Adresarea implicita

Adresarea implicita presupune ca adresele operanzilor sunt indicate in mod implicit de catre instructiune; nu se poate schimba localizarea operanzilor, care in majoritatea cazurilor se afla in registri. De exemplu, la DSP16xx (AT&T), toate instructiunile de inmultire folosesc operanzii din registrii de intrare ale multiplicatorului, X si Y, si depun rezultatul in registrul produs P. Faptul ca programatorul a folosit o instructiune de inmultire indica in mod implicit localizarea operanzilor. Astfel, desi sintaxa folosita de limbajul de asamblare al AT&T pentru operatia de inmultire este

P = X * Y

aceasta ar fi putut fi scrisa mai simplu drept

*

deoarece programatorul nu poate specifica alti registro in care se afla operanzii. Sintaxa AT&T se justifica totusi prin faptul ca ea prezinta explicit acesti registri, facilitand intelegerea codului scris in limbaj de asamblare

6.2 Adresarea imediata

In cazul adresarii imediate, operandul este inclus in cuvantul instructiune sau intr-un cuvant separat. De exemplu, in limbajul de asamblare al familiei ADSP-21xx de la Analog Devices, adresarea imediata poate fi utilizata pentru incarcarea unei constante intr-un registru, cum ar fi

AX0 = 1234

care specifica incarcarea constantei 1234 in registrul AX0. Este de remarcat faptul ca in acest exemplu valoarea constanta este specificata prin adresare imediata, in timp ce registrul destinatie, AX0, este specificat folosind adresarea directa la registru.

Prin intermediul adresarii imediate, cuvinte de date de lungime mica (de obicei jumatate din lungimea instructiunii sau mai mici) se pot coda in cuvantul instructiune, rezultand instructiuni codate cu un singur cuvint. Datele care au format mai mare trebuie plasate in codul instructiunii intr-un cuvant separat. In acest caz procesorul trebuie sa citeasca doua cuvinte din memoria de program inainte ca instructiunea sa poata fi executata, incetinind executia programului.

6.3 Adresarea directa la registru

In acest caz, datele ce trebuie adresate se afla intr-un registru. Programatorul specifica registrul ca parte a instructiunii. De exemplu folosind limbajul de asamblare de la familia TMS320C3x a Texas Instruments, instructiunea

SUBF R1, R2

scade valoarea aflata in registrul R1 din cea aflata in registrul R2 si plaseaza rezultatul in registrul R2. Diferenta dintre adresarea directa la registru si cea implicita (discutata mai sus) consta in posibilitatea programatorului de a specifica registrii utilizati, in timp ce in cazul adresarii implicite acestea sunt specificate de intructiune si nu pot fi schimbate. De exemplu instructiunea de scadere a TMS320C3x prezentata mai sus ar fi putut folosi oricare doi registri din cele opt registe existente pe cip. Acest mod de adresare este foarte important in procesoarele la care operanzii pentru instructiunile aritmetice provin din registri si nu din memorie.

6.4 Adresarea directa la memorie

In cazul adresarii directe la memorie (denumita si *adresare absoluta*), datele care trebuie adresate se afla in memorie la o adresa codata in cuvantul instructiunii. De exemplu, folosind limbajul de asamblare al ADSP-21xx (Analog Devices):

AX0 = DM(1000)

are drept efect incarcarea in registrul AX0 a datelor de la locatia 1000.

La fel ca in cazul adresarii imediate, adresele cu format mic pot fi codate in cuvantul de instructiune, rezultand o instructiune de un singur cuvant. Adresele mari trebuie plasate intr-un cuvant separat de cel al instructiunii. In acest caz procesorul trebuie sa citeasca doua cuvinte din memoria de program inainte ca instructiunea sa poata fi executata, incetinind executia programului.

6.5 Adresarea indirecta cu registru

La adresarea indirecta cu registru datele adresate se afla in memorie la o adresa indicata de un registru, asa cum este ilustrat in Figura 6-1. Unele procesoare au un grup special de registri (denumite *registri de adrese*) rezervate stocarii adreselor; alte procesoare au registri de uz general care pot stoca atat date cat si adrese. Adresarea indirecta cu registru este, pentru doua motive, cel mai folosit mod de adresare de pe majoritatea procesoarelor DSP. In primul rand acest mod de adresare este foarte potrivit utilizarii de tablouri de date, foarte raspandite in aplicatiile DSP. In sectiunea dedicata adresarii indirecte la registru cu pre- si post-incrementare se va detalia acest aspect. In al doilea rand, adresarea indirecta cu registru este eficienta din punct de vedere al setului de instructiuni: permite o adresare puternica, flexibila atunci cind se utilizeaza doar putini biti dintr-un cuvant instructiune.

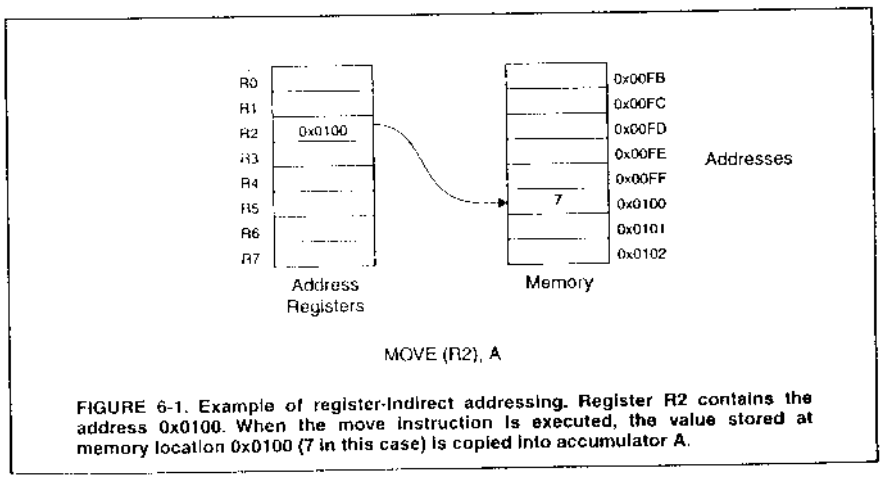


FIGURE 6-1. Example of register-indirect addressing. Register R2 contains the address 0x0100. When the move instruction is executed, the value stored at memory location 0x0100 (7 in this case) is copied into accumulator A.

Drept exemplu de adresare indirecta cu registru se poate considera instructiunea pentru DSP32xx (AT&T)

$$A0 = A0 + *R5$$

care are ca efect adunarea valorii stocate la locatia de memorie indicata de continutul registrului R5 la valoarea stocata in registrul acumulator A0. Rezultatul este depus in registrul acumulator A0. In aceasta instructiune, operandul de intrare „*R5” este specificat folosind adresarea indirecta prin registru: R5 este folosit drept registru de adresa, acesta stocheaza adresa locatiei de memorie care contine operandul propriu-zis.

In limbajul de asamblare folosit de AT&T, simbolul „*...”, atunci cand este folosit in acest mod, are sensul de „continutul locatiei de memorie specificat de registrul …”. Acest sens este cunoscut programatorilor care folosesc limbajul de programare C. Alte procesoare au sintaxe diferite pentru adresarea indirecta cu registru; multe folosesc „(Rn)” pentru a desemna locatia de memorie indicata la registrul de adrese Rn.

Exista multe variante ale adresarii indirecte cu registru descrise in cele ce urmeaza.

6.6 Adresarea indirecta cu registru cu pre- si post-incrementare/decrementare

Deoarece multe aplicatii DSP implica accesarea blocurilor de date din memorie, procesoarele DSP includ un numar de moduri de adresare concepute pentru a accelera accesul la date aflate in locatii de memorie cu adrese consecutive. Clasa de moduri de adresare denumite adresare indirecta cu registru cu pre- si post-incrementare contin facilitatile de baza ale adresarii indirecte cu registru descrise mai sus, dar adauga operatii care schimba valoarea/valorile din registrii utilizati cum ar fi, de exemplu, incrementarea/decrementarea unei valori dintr-un registru de adrese. Modificarea poate avea loc inainte ca adresa sa fie utilizata pentru a accesa datele din memorie (pre-modificare) sau dupa ce adresa a fost utilizata pentru a accesa datele (post-modificare). Modificarea poate fi incrementare sau decrementare.

Cea mai comuna modificare efectuata asupra unui registru de adrese consta in adaugarea unui increment cu valoarea unu dupa ce registrul de adrese este folosit la accesarea operandului. Acest

proces poarta denumirea de post-incrementare cu unu. Acest mod de adresare constituie un mecanism simplu de a parcurge un tablou de date stocat in memorie la adrese succesive. De exemplu, instructiunea pentru DSP32xx

A0 = A0 + *R5++

Aduna valoarea din locatia de memorie indicata de R5 cu cea din acumulatorul A0 si apoi incrementeaza valoarea din registrul de adrese R5 astfel incat sa specifice locatia de memorie urmatoare.

In mod similar, instructiunea

A0 = A0 + *R5--

are ca efect decrementarea valorii registrului R5 pentru a indica locatia de memorie anterioara dupa ce operatia de adunare a fost efectuata.

Unele procesoare au posibilitatea de a adauga sau scadea valori diferite de unu registrului de adrese. De exemplu, instructiunea procesorului DSP32xx

A0 = A0 + *R5++R17

aduna valoarea din locatia de memorie indicata de registrul R5 cu cea din acumulator si apoi incrementeaza valoarea din registrul de adrese R5 cu valoarea stocata in registrul R17. Deoarece valoarea din R17 poate fi negativa, acelasi format de instructiune poate fi folosit si pentru a decrementa valoarea stocata intr-un registru de adrese. La anumite procesoare exista un grup de registri dedicati stocarii valorilor de pre- si post-incrementare. In alte procesoare valoarea de incrementare poate fi stocata intr-un registru de uz general. Registrul care contine valoarea de incrementare (R17 in acest caz) se numeste registru pentru offset (Nota: unii producatori folosesc o terminologie diferita. De exemplu, Analog Devices denumeste acesti registri “registri modificatori.)

Un exemplu de mod de adresare cu pre-decrementare oferit de Motorola - DSP5600x:

MOVE X:-(R0), A1

care decrementeaza continutul registrului R0 inainte de utilizarea acestuia spre a indica la locatia de memorie continand valoarea ce urmeaza a fi mutata in registrul acumulator A1. Pre-incrementarea necesita de obicei un ciclu instructiune in plus pentru actualizarea registrului de adrese inaintea folosirii acestuia la adresarea memoriei.

6.7 Adresarea indirecta cu registru cu indexare

La adresarea indexata, valorile stocate in doi registri pentru adresa (sau intr-un registru pentru adresa si o constanta) sunt adunate pentru a rezulta adresa efectiva ce va fi utilizata la adresarea memoriei. Diferenta intre adresarea indirecta cu indexare si adresarea indirecta cu premodificare (discutata anterior) consta in faptul ca la indexare valorile stocate in registri nu sunt modificate. De exemplu, instructiunea pentru DSP5600x

MOVE Y1, X:(R5 + N6)

are ca efect mutarea valorii din Y1 la locatia de memorie din spatiul de adresare X a carei adresa este suma valorilor din registrii R5 si N6. R5 si N6 nu sunt modificate. Indexul este stocat in N6 care poarta denumirea de registru index. (Nota: unii producatori de procesoare folosesc termenul de „adresare indexata” pentru a denumi adresarea indirecta cu registru cu pre- si post-incrementare cu o valoare diferita de unu. La aceste procesoare, registrii denumiti anterior registri offset sunt uneori numite *registri index*.)

La unele procesoare valoarea index poate fi furnizata ca o valoare imediata codata in instructiune. De exemplu, instructiunea procesorului Texas Instruments, TMS320C3x

LDI *-AR1(1), R7

calculeaza o adresa efectiva scazand unu din valoarea registrului adresa AR1 si apoi muta datele din memorie la locatia indicata de adresa efectiva din registrul R7. Continutul registrului AR1 nu este modificat.

Adresarea indexata este utila atunci cand acelasi segment de cod este folosit cu mai multe seturi de date. Registrul index poate fi utilizat sa indice inceputul setului de date in memorie, iar registrii de adrese obisnuiti pot fi pre- sau post-modificate pentru a avansa prin setul de date in modul dorit. Cand programul este gata sa opereze asupra urmatorului set de date, trebuie modificata numai valoarea din registrul index.

Adresarea indexata este de asemenea utila in compilatoare la transmiterea argumentelor subrutinelor in/din stiva. Compilatoarele folosesc de obicei unul din registrii de adresa drept indicator de varf de stiva. De fiecare data cand o subrutina este apelata, indicatorul de varf de stiva este modificat in asa fel incat adresa la care indica sa contina varful de stiva anterior. Aceasta adresa contine numarul de argumente transmise rutinei, iar adresele urmatoare contin chiar aceste argumente. In acest mod subrutinele pot accesa argumentele lor intr-un mod simplu si unitar. De exemplu, daca registrul de adrese AR1 este folosit ca indicator de varf de stiva, atunci la TMS320C3x, instructiunea

LDI *+AR1(2), R0

poate fi folosita de o subrutina pentru a copia primul argument in registrul R0. Subrutina propriu-zisa nu depinde de locatia exacta in memorie a parametrilor sai; nu este necesar sa stie decat unde se afla acestia relativ la varful de stiva.

6.8 Adresarea indirecta cu registru cu aritmetica a adreselor modulo

In acest paragraf vom introduce modul de adresare modulo exemplificind cea mai comuna aplicatie a acestui mod de adresare, implementarea unui buffer circular. Multe aplicatii DSP necesita implementarea unor buffere. Un buffer de date este o portiune de memorie utilizata la a stoca datele provenite dintr-o sursa din exteriorul chipului sau de la o procesare anterioara pana cand procesorul este gata sa le prelucreze. La sistemele in timp real unde alocarea dinamica a memoriei este

inacceptabil de scumpa, programatorul trebuie sa determine cantitatea maxima de date pe care un buffer trebuie sa o memoreze si sa rezerve o portiune de memorie pentru acesta. Aceste buffere folosesc de obicei tehnica “first in – first out” (FIFO) care consta in citirea datelor din buffer in ordinea in care au sosit.

La implementarea miscarii datelor in buffer, programatorul foloseste doi pointeri, care sunt stocati in registri sau in memorie: un pointer de citire si unul de scriere. Pointerul de citire indica locatia de memorie unde se afla valoarea urmatoare ce va a fi citita, iar pointerul de scriere indica locatia de memorie unde va fi depusa urmatoarea valoare sosita, dupa cum este ilustrat in figura 6.2 (a). De fiecare data cand se efectueaza o operatie de citire sau de scriere, pointerii de citire sau de scriere avanseaza, iar programatorul trebuie sa verifice daca s-a ajuns la ultima locatie a bufferului. Testul daca pointerul a ajuns la sfarsitul bufferului dupa fiecare operatie si resetarea acestuia daca a ajuns consuma mult timp. La sistemele ce utilizeaza masiv bufferele aceasta poate fi o cauza de scadere semnificativa a performantei.

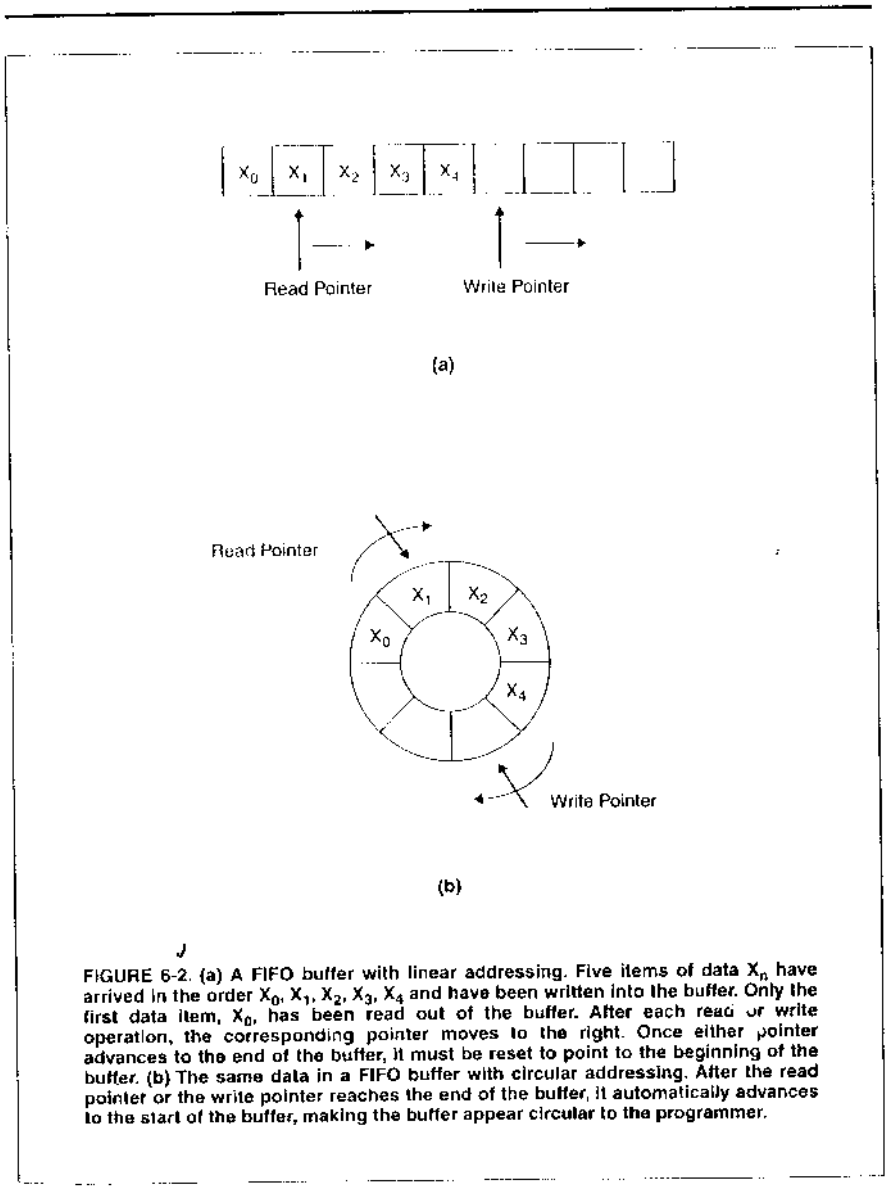
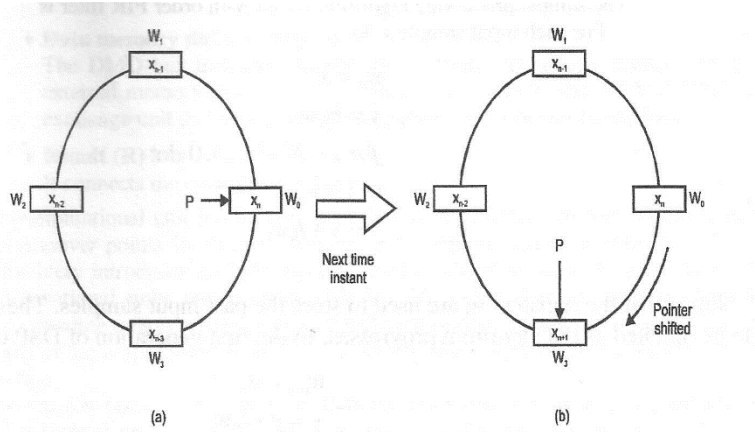


FIGURE 6-2. (a) A FIFO buffer with linear addressing. Five items of data X_n have arrived in the order X_0, X_1, X_2, X_3, X_4 and have been written into the buffer. Only the first data item, X_0, has been read out of the buffer. After each read or write operation, the corresponding pointer moves to the right. Once either pointer advances to the end of the buffer, it must be reset to point to the beginning of the buffer. (b) The same data in a FIFO buffer with circular addressing. After the read pointer or the write pointer reaches the end of the buffer, it automatically advances to the start of the buffer, making the buffer appear circular to the programmer.

Pentru a inlatura acest dezavantaj, multe DSP-uri au un mod de adresare special care le permite sa efectueze automat aceste verificari dupa fiecare operatie de calcul a adreselor bufferului si sa ajusteze pointerul la inceputul bufferului daca este necesar. Aceasta capabilitate se numeste *adresare modulo* sau *adresare circulara*. Principiul functionarii buffer-ului circular este prezentat in figura urmatoare.



Buffer-ul contine in fiecare moment ultimele patru esantioane care trebuie inmultite cu constantele w_0, w_1, w_2 si respectiv w_3. La momentul *n* pointerul P acceseaza ultima data sosita, x_n. La momentul urmator (n+1) pointerul este deplasat la adresa ocupata de esantionul x_{n-3} (cel mai vechi esantion care trebuie inlocuit cu esantionul x_{n+1} . Folosind bufferul circular, prin ajustarea automata a pozitiei pointerului, cantitatea de date care trebuie deplasata este minimizata.

Termenul *modulo* se refera la aritmetica modulo, unde numerele sunt limitate la un anumit interval. Acest proces este similar aritmeticii utilizate la ceas, care este bazat pe un ciclu de 12 ore. Atunci cand rezultatul calculelor depaseste valoarea maxima este ajustat prin scaderea repetata a valorii reprezentative maxime pana cand rezultatul se incadreaza in domeniul specificat. De exemplu, ora 14 este ajustata la ora 2 (14 modulo 12).

Atunci cand se utilizeaza aritmetica modulo, pointerii de citire si scriere (registrii de adresa) sunt actualizati folosind adresarea indirecta cu registru cu pre- si/sau post-incrementare. Unitatea de generare a adreselor din procesor foloseste aritmetica modulo atunci cand se calculeaza adresele noi creand impresia unei organizarii circulare a memoriei, asa cum este ilustrat in Figura 6.2 (b). Adresarea cu aritmetica modulo elimina necesitatea verificarii de catre programator a depasirii bufferului de catre pointerii de citire si scriere si de ajustare a acestora in cazul afirmativ. Ca efect, operatiile cu buffere sunt efectuate mult mai rapid, facand din adresarea modulo o capabilitate valoroasa in multe aplicatii.

Multe din procesoarele DSP recente asigura suport pentru adresarea cu aritmetica modulo. Totusi, nivelul de suport oferit si mecanismul folosit pentru control variaza de la procesor la procesor. Aceste implementari sunt discutate in paragraful urmator.

Programatorul controleaza adresarea modulo in doua moduri:

- In primul mod, lungimea bufferului circular este incarcata intr-un registru special, denumit registru modificator sau registru modulo. Un procesor poate avea un singur registru modulo sau poate avea cateva. Fiecare registru modulo este asociat cu unul sau mai multi registri adresa; de fiecare data cand se incarca o valoare in registrul modulo, registrul adresa asociat utilizeaza automat aritmetica modulo. Deoarece registrul modificator contine doar lungimea bufferului, nu si adresa de inceput, mecanismul de adresare modulo trebuie sa faca unele presupuneri despre adresa de start a bufferului. De obicei, bufferele circulare trebuie sa inceapa la granita segmentelor de *k*-cuvinte, unde *k* este cea mai mica putere a lui 2 care este egala sau mai mare cu dimensiunea bufferului circular. De exemplu, un buffer circular de 48-cuvinte trebuie de obicei sa fie la o granita a unui segment de 64-cuvinte, deoarece 64 este cea mai mica putere a lui 2 egala sau mai mare cu 48. Daca ne imaginam impartirea spatiului de adrese a unui procesor in blocuri de *k*-cuvinte, incepand de la adresa 0, atunci avem o granita de segment de *k*-cuvinte la fiecare adresa dintre doua segmente de k-cuvinte consecutive. De exemplu, un buffer circular cu lungimea 256 poate incepe la adresa 0, 256, 512 sau orice alta adresa multiplu de 256. Procesoarele care implementeaza acesta forma a modului de adresare modulo (sau una similara) includ Texas Instruments - TMS320C3x si TMS320C4x, si procesoare de la Motorola, Analog Devices, NEC si DSP Group.
- Un mod alternativ de tratare a adresarii modulo foloseste registri de start si sfarsit pentru a stoca adresele de start si sfarsit ale fiecarui buffer circular. Pe unele procesoare, adresarea cu aritmetica modulo este apoi folosita cu orice registru de adresare care acceseaza zona de memorie delimitata de adresele din registrii de start si sfarsit. Pe alte procesoare unui anumit buffer circular ii este asociat numai un registru de adresare. Procesoare care folosesc registri de start/sfarsit sunt DSP16xx (AT&T) si TMS320C5x (TI).

Dupa cum a fost sugerat mai sus, diverse procesoare au un numar diferit de buffere circulare active cu adresare modulo. De exemplu, AT&T - DSP16xx suporta adresarea cu aritmetica modulo doar pe un singur buffer, deoarece are doar un registru de start si unul de sfarsit. Procesorul Texas Intruments - TMS320C5x suporta doua, Motorola - DSP561xx suporta patru, iar Motorola - DSP5600x si toate procesoarele de la Analog Devices suporta opt.

6.9 Adresarea indirecta cu registru cu inversarea bitilor

Acest mod de adresare este folosit numai in anumite imprejurari foarte speciale. Unele aplicatii DSP folosesc intens algoritmi de transformare Fourier rapida (FFT). Algoritmii FFT sunt algoritmi de calcul rapid a transformatei Fourier discrete. O categorie importanta de algoritmi FFT sunt algoritmii in baza 2 (radix -2). Exista doua categorii de algoritmi in baza 2: cu decimare in timp si cu decimare in frecventa. Aceasta categorie de algoritmi prezinta insa dezavantajul ca ori folosesc date de intrare intr-o ordine inversata, ori produc datele de iesire intr-o ordine inversata. Aceasta situatie impune rearanjarea datelor in ordinea fireasca la un moment dat.

Tipul de inversare depinde de variantele algoritmului FFT folosit. Implementarea FFT in baza 2 (FFT radix-2), necesita reordonarea datelor intr-un mod simplu denumit ordine inversa a bitilor. Termenul de inversare a bitilor se refera la observarea faptului ca daca valorile de iesire ale unui numarator binar sunt scrise in ordine inversa (bitul cel mai putin semnificant primul) secventa rezultanta se va potrivi cu iesirea inversata a datelor de iesire a algoritmului FFT. Aceasta situatie este ilustrata in Fig. 6.3.

Deoarece FFT este un algoritm important in multe aplicatii DSP, multe procesoare includ hardware specializat in unitatea de generare a adreselor pentru a facilita generarea de secvente de adrese cu bitii inversati pentru rearanjarea rezultatelor FFT. De exemplu, ADSP-210xx (Analog Devices) ofera un mod de adresare cu bitii inversati care este activat prin setarea unui bit in registrul de control. Cand procesorul se gaseste in acest mod, iesirii unuia din registrii de adrese ii sunt inversati bitii inainte de a fi aplicata pe magistrala adreselor memorie.

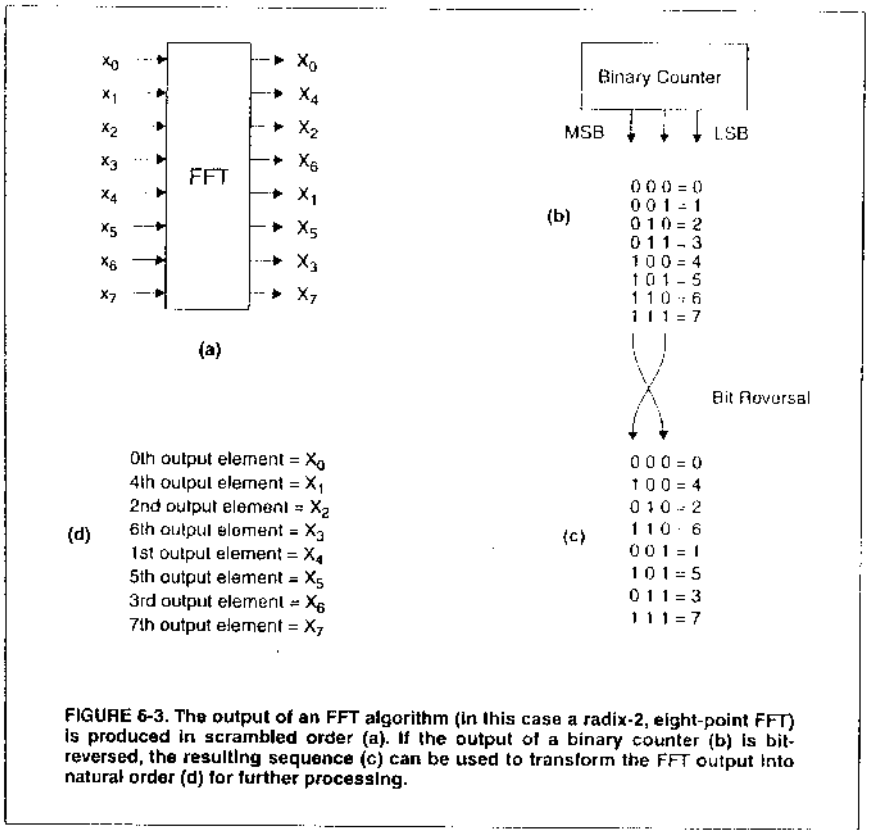


FIGURE 6-3. The output of an FFT algorithm (in this case a radix-2, eight-point FFT) is produced in scrambled order (a). If the output of a binary counter (b) is bit-reversed, the resulting sequence (c) can be used to transform the FFT output into natural order (d) for further processing.

Fig. 6.3

O abordare alternativa la implementarea adresarii cu inversare a bitilor consta in utilizarea aritmeticii cu transport invers. In cazul aritmeticii cu transport invers, unitatea de generare a adreselor inverseaza directia in care se propaga bitii de transport cand un increment este adaugat valorii dintr-un registru adresa. Daca este activata aritmetica cu transport invers in AGU si programatorul pune la dispozitie adresa de baza si valoarea incrementului cu ordinea bitilor inversata, adresa rezultanta va

fi cu ordinea bitilor inversata. Aritmetica cu transport invers este utilizata, de exemplu, in DSP32C si DSP32xx (AT&T).

6.10 Modurile de adresare scurte

Dupa cum a fost ilustrat anterior in acest capitol prin cateva exemple, un neajuns al catorva moduri de adresare este acela ca in general instructiunea si adresa necesita doua cuvinte de program pentru memorare, ceea ce mareste dimensiunea programului si incetineste executia. Unele procesoare ofera versiuni scurte ale unor moduri de adresare. Aceste versiuni scurte includ intr-un singur cuvant atat instructiunea cat si adresa, dar pe seama unor restrictii la nivelul adreselor.

a) Adresarea scurta cu date imediate

In modul de adresare cu date imediate operandul propriu-zis (si nu adresa unde este stocat) este codat in cuvantul instructiune sau intr-un cuvant de date ce urmeaza cuvantului instructiune. In sintaxa limbajului de asamblare al Motorola DSP5600x, de exemplu, adresarea imediata poate fi folosita la incarcarea unei constante intr-un registru, cum ar fi

MOVE #1234, A

care incarca valoarea constanta in registrul A. In general instructiunea necesita doua cuvinte de memorie de program: unul pentru stocarea codului operatiei de incarcare si unul pentru stocarea datelor imediate. Totusi, daca datele imediate sunt mici, ar fi posibila includerea instructiunii si a datelor intr-un singur cuvant. In cazul Motorola DSP5600x, instructiunile cu date imediate in forma de mai sus pot fi codate intr-un singur cuvant de memorie de program daca datele au 12 biti sau mai putin.

b) Adresarea scurta directa la memorie

Din discutia anterioara asupra adresarii directe la memorie, datele ce urmeaza a fi adresate se afla in memorie, si adresele la care acestea se afla sunt codate in cuvantul instructiune sau intr-un cuvant care urmeaza instructiunii. De exemplu, la Motorola DSP5600x, instructiunea in limbaj de asamblare

MOVE $1000, A

are ca efect incarcarea datelor aflate la adresa 1000 in registrul A. In acest procesor Motorola, adresarea scurta directa la memorie permite codarea in instructiune a adreselor de pana la sase biti lungime. Aceasta solutie economiseste un cuvant de memorie (si un ciclu instructiune) comparativ cu varianta de adresare directa la memorie descrisa anterior.

In unele cazuri, procesoarele pot pune la dispozitie si alte versiuni de adresare scurta directa la memorie care adauga un offset implicit la adresa scurta. De exemplu, DSP5600x foloseste un mod

de adresare scurta directa la memorie pentru adresarea registrilor I/O care sunt mapate pe ultimele 64 locatii de memorie.

6.11 Adresarea directa la memorie paginata

In cazul adresarii directe la memorie paginata, un registru special stocheaza numarul paginii (sectiuni) de memorie care este adresata. Astfel o anumita instructiune poate adresa direct un cuvant in interiorul paginii specificand locatia relativa a cuvantului in pagina. Pentru adresarea unui cuvant din afara paginii, registrul de pagina trebuie actualizat (sau se poate utiliza un alt registru de pagina daca sunt mai multe disponibile).

De exemplu, desi procesoarele Texas Instruments TMS320C2x si TMS320C5x pot adresa pana la 64 Kcuvinte de memorie de date, instructiunile care folosesc adresarea scurta directa la memorie nu folosesc direct o adresa de 16 biti. Instructiunile ce folosesc acest mod de adresare au 7 biti de adrese, suficienti pentru a selecta un cuvant din 128 (2^7). Procesorul imparte memoria in pagini de 128 cuvinte fiecare. Programatorul selecteaza o pagina incarcand numarul paginii in registrul indicator de pagina. Instructiunile care urmeaza pot executa accesari in interiorul paginii de 128 cuvinte fara a mai fi nevoie de un al doilea cuvant de memorie de program care sa specifice adresa. Desigur, aceasta tehnica este eficienta doar daca programatorul poate organiza programul astfel incat da aiba loc un numar semnificativ de accesari in interiorul unei pagini inainte de a fi nevoie de utilizarea altei pagini.